

10 ACID, Transactions

▼ What is a Transaction?



A **transaction** is a set of operations (e.g., SQL statements) considered one **undividable** unit of work.

- It should always either **completely succeed** (commit) or **completely fail** (rollback).
- **Example:** Transferring money between two accounts involves two update operations:

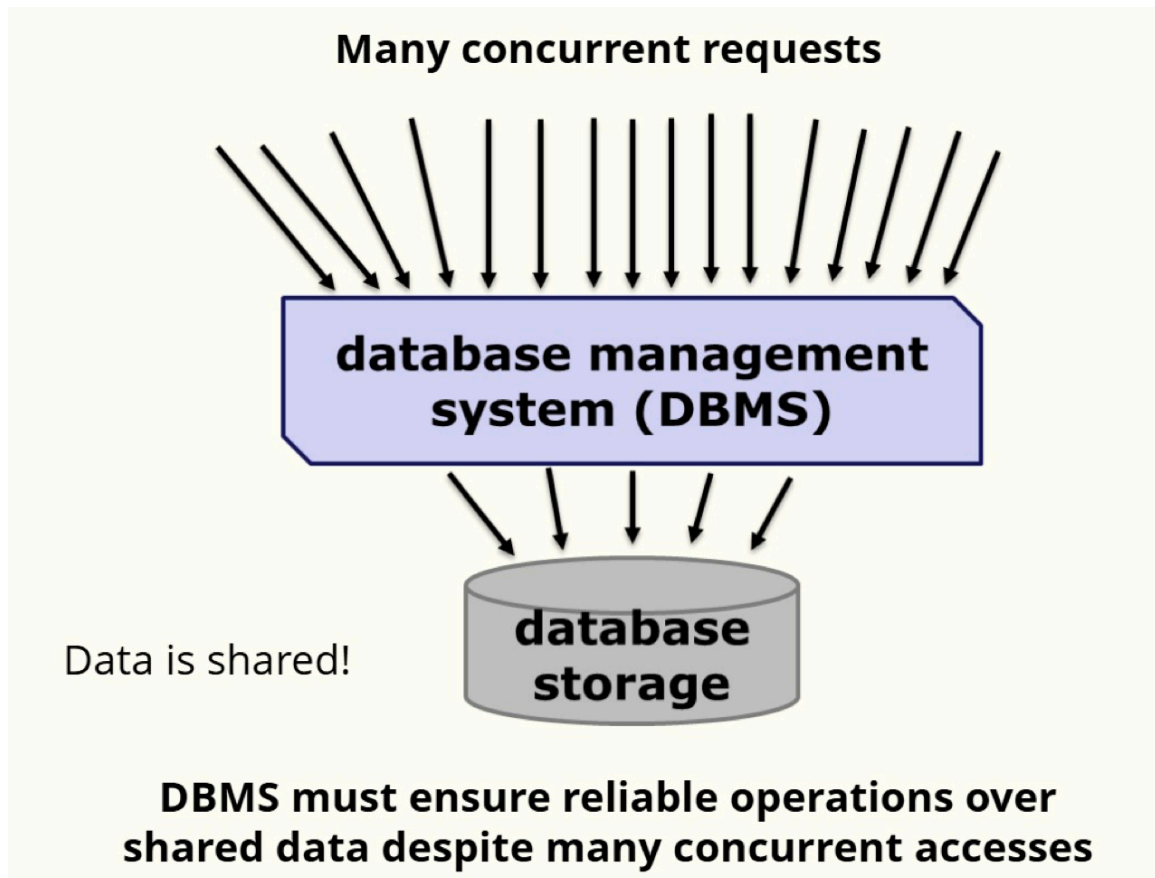
```
BEGIN TRANSACTION;  
<SQL Statement 1>;  
<SQL Statement 2>;  
COMMIT; or ROLLBACK;
```

- **COMMIT** if the transaction runs as expected
- **ROLLBACK** if you encounter an error
 - Can also be used for testing

```
<begin_transaction>  
UPDATE account SET balance = balance - :amount WHERE accountnumbe  
r = :account_to_debit;  
UPDATE account SET balance = balance + :amount WHERE accountnumbe  
r = :account_to_credit;  
<end_transaction>
```

▼ Why do we need transactions?

- Many things can crash:



▼ The Transaction Lifecycle

- **States:**
 - **Active:** The transaction is being executed.
 - **Committed:** The transaction has successfully completed.
 - **Aborted:** The transaction has failed and must be rolled back.
- **Start and End:**
 - Begins with `<begin_transaction>` or an implicit first SQL statement.
 - Ends with `<end_transaction>` followed by either `COMMIT` or `ROLLBACK`.

▼ Key DBMS Components

- **Transaction Manager:** Coordinates transaction execution.
- **Scheduler:** Determines the order of operations and manages concurrency.
- **Stored Data Manager:** Interfaces with the database files.

- **Buffer Manager:** Manages in-memory data buffers.
- **Recovery Manager:** Ensures recovery from failures.

▼ Types of Failures

- **System failure:** DBMS crash or OS failure.
- **Media failure:** Disk crash or corruption.
- **Transaction failure:** Application-level error (e.g., division by zero).

▼ Concurrency Problems

▼ 1. Lost Update Problem

- Two transactions read the same value and update it.
- One update is overwritten by the other.
- **Example:**
 - T_1 and T_2 both read balance = 100.
 - T_1 writes $100 - 50 = 50$.
 - T_2 writes $100 + 120 = 220$.
 - Result: T_1 's update is lost.

▼ 2. Uncommitted Dependency (Dirty Read)

- A transaction reads data from another that hasn't committed.
- If the first transaction aborts, the read data was invalid.

▼ 3. Inconsistent Analysis (Non-repeatable Read)

- A transaction reads the same data item twice and gets different values due to other committed transactions in between.

▼ Scheduling and Serializability

- **Schedule:** Order of execution of transaction operations.
- **Serial schedule:** Transactions execute one after the other.
- **Serializable:** Equivalent to some serial schedule, ensures correctness.

▼ Optimistic vs. Pessimistic Scheduling

- **Optimistic:**
 - Assumes low conflict.
 - Transactions execute freely, check for conflicts at commit.
 - Costly rollbacks if conflicts exist.
- **Pessimistic:**
 - Assumes high conflict.
 - Locks data before use to prevent interference.
 - Less rollback, but can reduce concurrency.

▼ Locking and Protocols

Write-Ahead Logging (WAL Protocol)

1. Before any changes are written to disk we **force the corresponding record to disk**
 - Ensures **atomicity**
2. Before a transaction is committed we **force all log records for the transaction to disk**
 - Ensures **durability** 🍷 📱

▼ Locks General

- A lock is a **right to perform operations** on a db element
- **Only one** transaction may hold a lock on **an element at any time**

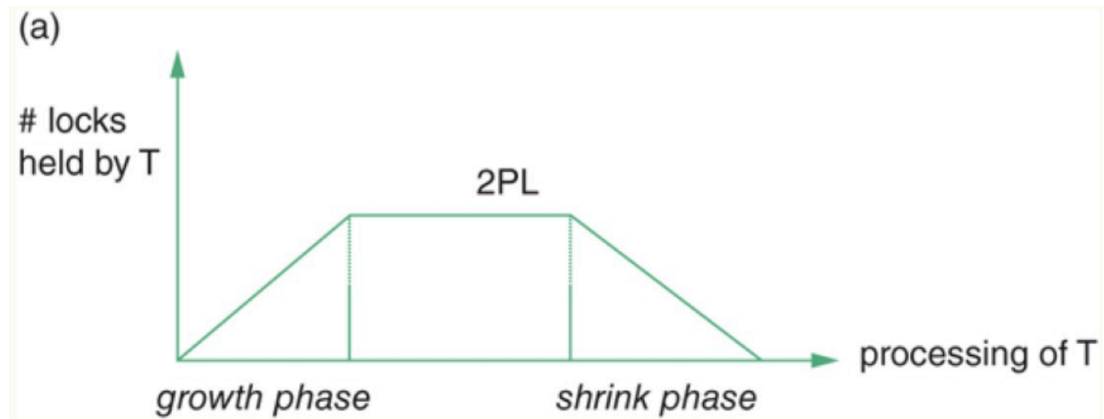
▼ Lock Types

- **Shared Lock (S-lock):** For reading
- **Exclusive Lock (X-lock):** For writing

▼ Two-Phase Locking (2PL)

- **Growth phase:** Acquire locks.

- **Shrink phase:** Release locks.



- **Variants:**
 - *Rigorous 2PL*: All locks held until commit.
 - *Static/Conservative 2PL*: All locks acquired before starting.

▼ Multiple Granularity Locking (MGL)

- Locks at different object levels: tuple, table, tablespace.
- Uses intention locks:
 - **IS-lock**: Intention shared.
 - **IX-lock**: Intention exclusive.
 - **SIX-lock**: Shared and intention exclusive.
- Locks acquired top-down, released bottom-up.

▼ Deadlocks and Cascading Rollbacks

- **Deadlock**: Two transactions wait for each other's locks.
 - *Resolution*: Victim selection (abort one transaction).
- **Cascading Rollback**: One abort leads to others that depended on it to also abort.

▼ Isolation Levels

Level	Description	Risk
-------	-------------	------

Read Uncommitted	Reads uncommitted changes	Dirty reads
Read Committed	Only reads committed changes	Non-repeatable reads
Repeatable Read	Prevents updates during transaction	Phantom reads possible
Serializable	Fully isolated	No concurrency anomalies

▼ ACID Properties

Atomicity:

- A Transaction is "one operation"
- Each transaction runs to completion or no effect
- All-or-nothing.

Consistency:

- Each Transaction moved the DB from one consistent state to another
- From one valid state to another.

Isolation:

- No interference between concurrent transactions.
- Alone in the world

Durability:

- Committed changes persist despite failures.

▼ Distributed Storage Systems

▼ General

Workload sharing - balance

- Increased performance
- Better resource utilization

Redundancy (replicas)

- Failure handling

- High Availability

Exercises